

GNN drug discovery project

Background

TDC = dataset (data repository) for drug discovery

-> For single drugs (molecule + property)

https://tdcommons.ai/multi_pred_tasks/drugsyn/

look at drug comb

Usually RDKit is then used - a python package - to convert molecules

Graph we input to GNNs - atoms list and their connectivity matrix between them

On top of that one can use some "pytorch geometric" -> converts graphs to pytorch

Meetings with Savir

1st meeting - April 2026

- Smiles contains 3D data, all the 2D and almost all 3D. Including Krality, etc.
 - Genes tell us if compound worked on skin cell or on כליה cell for instance
 - Problems: tens/hundreds of entries to represent molecules vs thousands for genes. Also, genes data unorganized.
 - We can use interactions between genes from literature and different experiments. -> we can use it as another graph. It can help us solve either for 1 drug or 2 drugs.
 - The synergy score based on many elements and matrixes/metrics (?). We could work with how it affects the genes, but we don't have that info unfortunately. We only have the "control" group. We only have "an ID" for the celline (not different drugs, etc.).

Sources

DeepSynergy, baseline for the current MLP-based synergy predictor

Preuer, K., Lewis, R. P. I., Hochreiter, S., Bender, A., Bulusu, K. C., and Klambauer, G. DeepSynergy: predicting anti-cancer drug synergy with Deep Learning.

<https://academic.oup.com/bioinformatics/article/34/9/1538/4747884>

<https://www.bioinf.jku.at/software/DeepSynergy/>

SOTA model Diagram

https://ars.els-cdn.com/content/image/1-s2.0-S1476927125003974-ga1_lrg.jpg

<https://www.sciencedirect.com/science/article/abs/pii/S1476927125003974>

<https://academic.oup.com/bioinformatics/article/34/8/1433/4657075>

Main reference implementation used to understand a stronger drug-synergy pipeline and where gene expression enters the model

Kuru, H. I., Tastan, O., and Cicek, A. E. MatchMaker: A Deep Learning Framework for Drug Synergy Prediction.

DrugComb

Zagidullin, B. et al. DrugComb: an integrative cancer drug combination data portal.

Baseline for autoencoder-style compression of high-dimensional omics features before synergy prediction

AuDNNsynergy.

Relevant next-stage references (network-informed or biologically structured encoders for gene-expression data).

PRODeepSyn and TranSynergy

Insights

Combine LLMs:

Query = what I need

Key = what exists

Value = actual info

—>

Query = drug / task context (what matters biologically)

Keys = genes

Values = gene expression levels

Q = f(drug_a, drug_b) # what we care about
K = gene embeddings # what exists
V = gene expression # actual signal

Directions

Gene expression: it describes the type of cell-line (ריקמה) - and it can show what type of cancer you have, and thus what medicine would be better. There are about 20k genes. There are knowledge graphs that show how one gene affects on the other, some call it protein on protein interaction graph. We can be given an input of such knowledge graph as context of a healthy person (how it should be)

In the dataset: 20k columns (of genes, cellines, taken via biopsy - from person with cancer). Each row is interaction between 2 diff molecules.

The goal - since current approaches take the 20k and compress it to 128-256 a lot of info is lost. They want to improve the decoder (compressor). Potentially by using info about interactions (healthy person vs sick to find anomalies)

Then they tried using diffusion (sum of nodes?), but since they're having scalars on vertices, the data is erased quite quickly

SO how to combine the context info correctly such that the prediction for synergy will be good.

Given 3 embeddings vectors - gene expression, for drug (molecule) A, for drug B. How do we infuse them? Currently people use 2 cross-attention blocks (where gene expression is added to the second one after)

- convert molecules
 - Graph - atoms list and their connectivity matrix between them
 - On top of that use some "pytorch geometric" -> converts graphs to pytorch

Dictionary

Data / Sources

- **TDC (Therapeutics Data Commons)** – benchmark datasets for drug discovery
- **DrugComb** – drug-drug synergy dataset (pairs of drugs + cell line + synergy score)

Molecule representation

- **SMILES** – string encoding of a molecule (input format)
- **RDKit** – converts SMILES → molecular graph (atoms + bonds)

Graph concepts

- **Graph (molecule)** – nodes = atoms, edges = bonds
- **Adjacency / edge_index** – connectivity between atoms
- **Node features** – atom properties (atomic number, valence, etc.)

Frameworks

- **PyTorch Geometric (PyG)** – builds GNNs on graphs (Data objects)

Models

- **GNN (GCN / GIN)** – encodes molecule graph → vector embedding
- **Embedding** – compressed vector representation (drug / cell line)

Biology

- **Gene expression** – ~20k-dimensional vector per cell line (activity of genes)
- **Cell line** – cancer sample (context where drugs act)
- **PPI (Protein-Protein Interaction)** – graph of gene/protein interactions
- **DTI (Drug-Target Interaction)** – which proteins a drug affects
- **DDI (Drug-Drug Interaction)** – interactions between drugs

Task

- **Drug synergy prediction** – given (drug A, drug B, cell line) → score

Modeling concepts

- **Fusion** – combining embeddings (drug A, drug B, gene expression)
- **Cross-attention** – lets one modality (e.g. genes) influence another (drugs)
- **Compression (encoder)** – reducing 20k genes → ~128 dims (lossy bottleneck)

Cellines

- CellLine column = gene expression representation of a cancer cell line (input to model)
- Each value = expression level of a specific gene (how active that gene is in that cell line)
- Index is consistent: index i = same gene across all rows (critical for ML)
- Raw gene space $\approx$ 20k–25k genes (dataset-specific; not all genes in biology)
- Observed structure in your data: CellLine contains **3 arrays**
 - ~23808 → raw gene expression (full measured genes)
 - ~3171 → filtered genes (selected subset, e.g., high variance / informative genes)

- ~627 → compressed / pathway-level features (aggregated biological signals)
- These multiple arrays likely represent **progressive compression levels**:
 - raw → filtered → biologically compressed
- Gene names are NOT included → currently index has no biological meaning (just position)
- To use biological networks (PPI, transcription), need mapping: index → gene (e.g., TP53)
- Gene expression values:
 - continuous (floats)
 - often preprocessed (log-transformed, normalized)
 - typical range ~2–15 after normalization
- DrugComb CSV columns:
 - Drug1_ID / Drug2_ID → drug names
 - Drug1 / Drug2 → SMILES (molecular structure)
 - Cell_Line_ID → name (e.g., 786-0)
 - CellLine → gene expression arrays (features)
 - CSS → combination sensitivity score
 - Synergy_ZIP / Bliss / Loewe / HSA → different synergy metrics
- ZIP score:
 - 0 synergy, <0 antagonism
- preferred target for ML
- Important distinction:
 - Cell_Line_ID = identifier (string)
 - CellLine = actual feature vector (what model uses)
- Key insight:
 - model does NOT see biology directly
 - it sees a high-dimensional vector representing gene activity
- Implication for research:
 - improving encoder = improving how this ~20k-dimensional signal is compressed into meaningful representation
 - current datasets already include multiple compression levels → useful for benchmarking
- Practical note:
 - different datasets may have different gene sets/order → pipelines must stay consistent within dataset

TODOs

Baseline

- ✓ What is Quantile normalization and FARMS (Factor Analysis for Robust Microarray Summarization) used for and can it be helpful to this research?
- ✓ Explain flow with blocks visuals how the program flows - which files responsible for what?
- ✓ 3 options for test set: exclude drug A/B that were used (only use ones that weren't use), or use celline that weren't used in train
- ✓ not too many epochs, too big LR etc.
- ✓ Use 10-cross validation (Even better- stratified cross validation) with 10 seeds for the random generator. But it takes a long time, so we could use 10K.
- ✓ Try predict WITH gene expression and WITHOUT gene expression to see if it helps (or makes it worse).
- ✓ Check current compression with the baseline I built
- ✓ You mentioned auto encoder is identity - which files shows it? what is the current architecture of the model? use block visuals.
- ✓ Prep for tomorrows meeting with Ronen : As I understand, the question we shall ask Which input dimensions does the prediction depend on, and can we map those dependencies back to genes?
But for current run we use PCA128. Auto-encoder is identity.

April - May meeting (OCA & new arch)

- replace manual embeddings (fingerprints) of smiles to GNN, LLM, or something (conduct deep research) that will allow reverse engineering to the atom level.
- Replace PCA with GNN, LLM, or something (conduct deep research) that will allow reverse engineering to the gene level.
- When RMSE and OCA works the same or better on new model - message ronen, learn how to reverse engineer.
- ✓ start oca on baseline (see that nothing makes it jump too much on the model). The OCA will tell us if we have local or global triggers.
- ✓ Add figures (regression, etc.) to notes below 📌
 - GNN would be 2 birds in one stone - would be easier to invoke on celline since we have on smiles, and LLM is autoregressive and sees previous
 - In the upcoming weeks we will reverse engineer the MLP to see what it looks at in the genes. Analysis methods include OCA - mask the input

and check the delta error that was created in output to see affects.
Could be to look at the neuron - etc., but that's problematic (for some reasons I forgot). Finally, could use attention mechanism, by lowering the "d" factor in the Query Keys product so that the result is noiser and the sigmoid has to work harder.

Journal

Tools we use

Data source / access:

- PyTDC

Core ML:

- PyTorch

Data handling:

- pandas
- numpy

Plots / artifacts:

- matplotlib

Classical ML / evaluation:

- scikit-learn
- scipy

Environment / runner:

- uv

Not used currently:

- RDKit

March/April 2026 - Literature leg work, learning, framework

Went over relveant literature - DeepSynergy, SOTA models, and other sources. See Sources section above.

- Dataset has only 59 unique cell lines, so PCA128 cannot contain 128 fully independent informative components.
- drug_and_cell_line split uses priority routing: test > val > train, and mixed rows are preserved rather than dropped.
- no-gene baseline is strong, so gene encoder claims must prove OOD

improvement, not only random-split improvement.

- Planned compression candidates:

1. z-score -> Var3k -> PCA512 -> AE128
2. z-score -> BioFilter1k-3k -> AE128
3. raw -> AE128, lower priority

- Neuron-level MLP interpretation is lower priority because hidden neurons are not stable semantic units.

- Attention/gating weights are not automatically explanations; they must be validated with perturbation/OCA.

17.4.2026 - Implementing DeepSynergy baseline

Using raw genetic cellline data from drugcomb csv DECREASES performance.

With genes model predicts constant -> no correlation -> Val Pearson/Spearman: nan/nan.

With ~23k cellline genes test MSE: 29.051481

With ~3k cellline genes test MSE: 29.046721

Without cellline genes test MSE: 20.250234

18.4.2026 - Baseline completion, random, drug, and cellline splits, and 10 fold CV

Filtered with PCA and top variance to 128 and MSE improved: 19.041908

Data compression pipeline: raw `CellLine[0]` = `23808` dimensions -> keep top `3000` highest variance -> z-score -> PCA

Compared additional runs with/without genes and with/without drug and cellline exclusion in train-validation-test split.

With genes worked better for 10 fold and random.

drug_and_cell_line - test-validation-train split excluding some celllines AND genes from train and validation.

Pick some celllines and genes to exclude from train -> pick from remainder to exclude from validation -> put remaining in test.

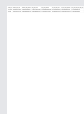
cell_line -> same only with cellline exclusion

drug -> same only with drug exclusion

random -> no exclusion. random splits

baseline_summary.csv

Text Document · 547 bytes

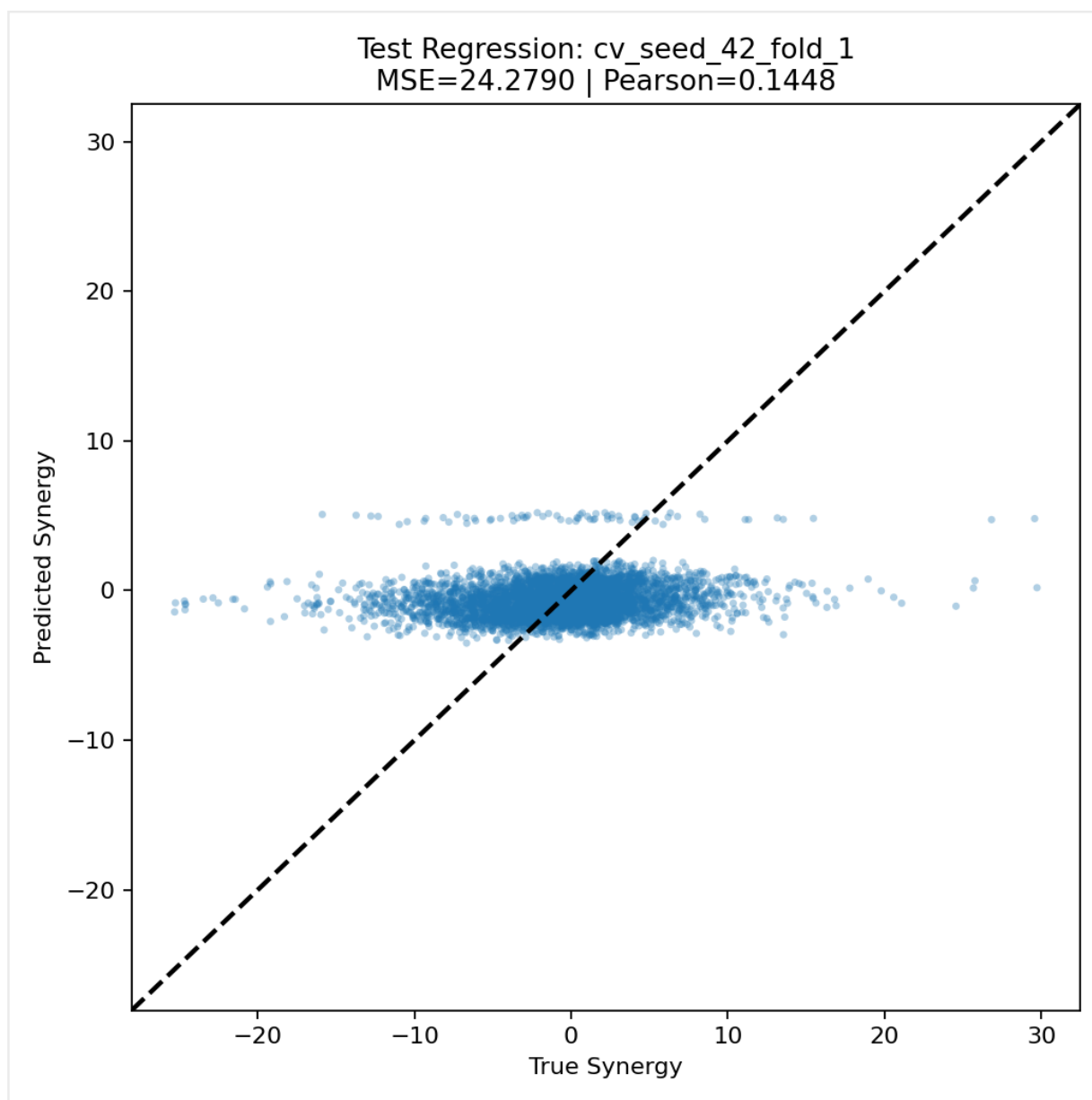


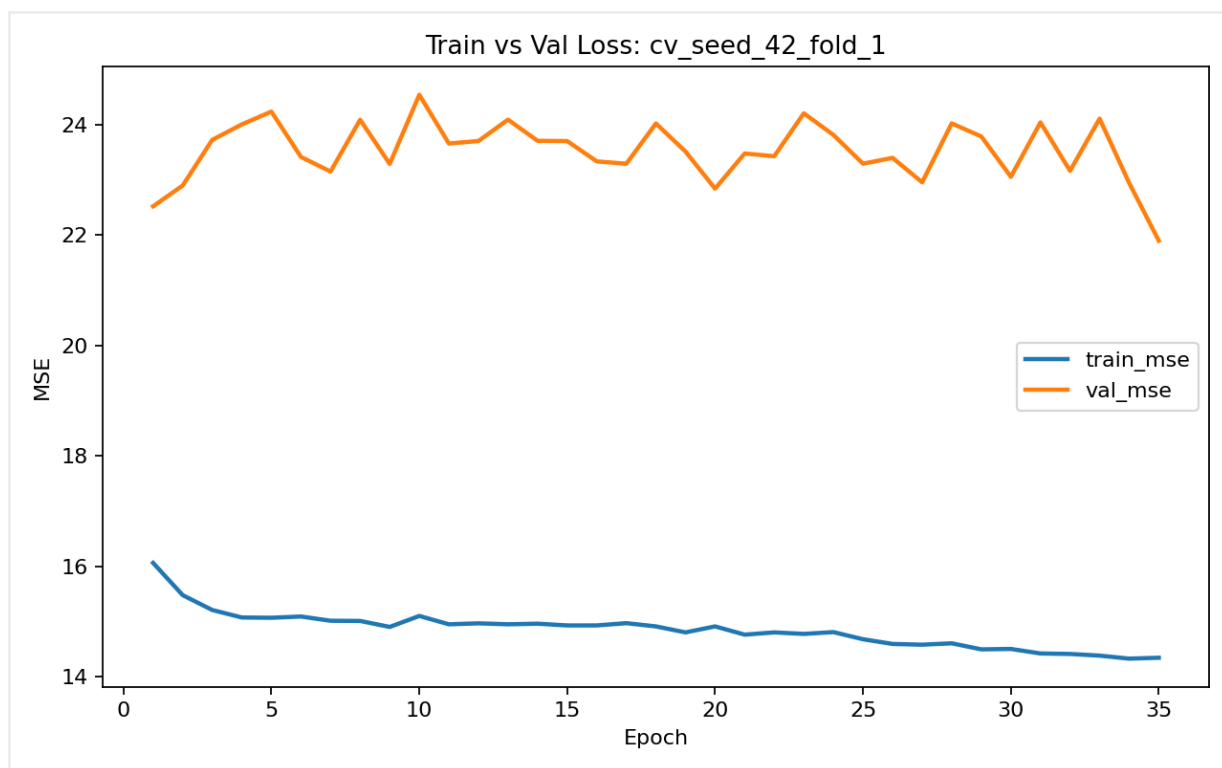
25.4 - Regression plots and training plots

Regression plots added for each fold in 10 fold CV. Results are as expected. Not covering all synergy score we have in the range, specifically the highest and lowest, but it's expected. They are focused more around the 0 but still spread out.

In addition to training-validation vs epochs plots. From the long_cv10_10k_pca128_drug_and_cell_line_practical, i.e., 10 fold CV with celline and drug exclusion (see 18.4 notes), we got the following results.

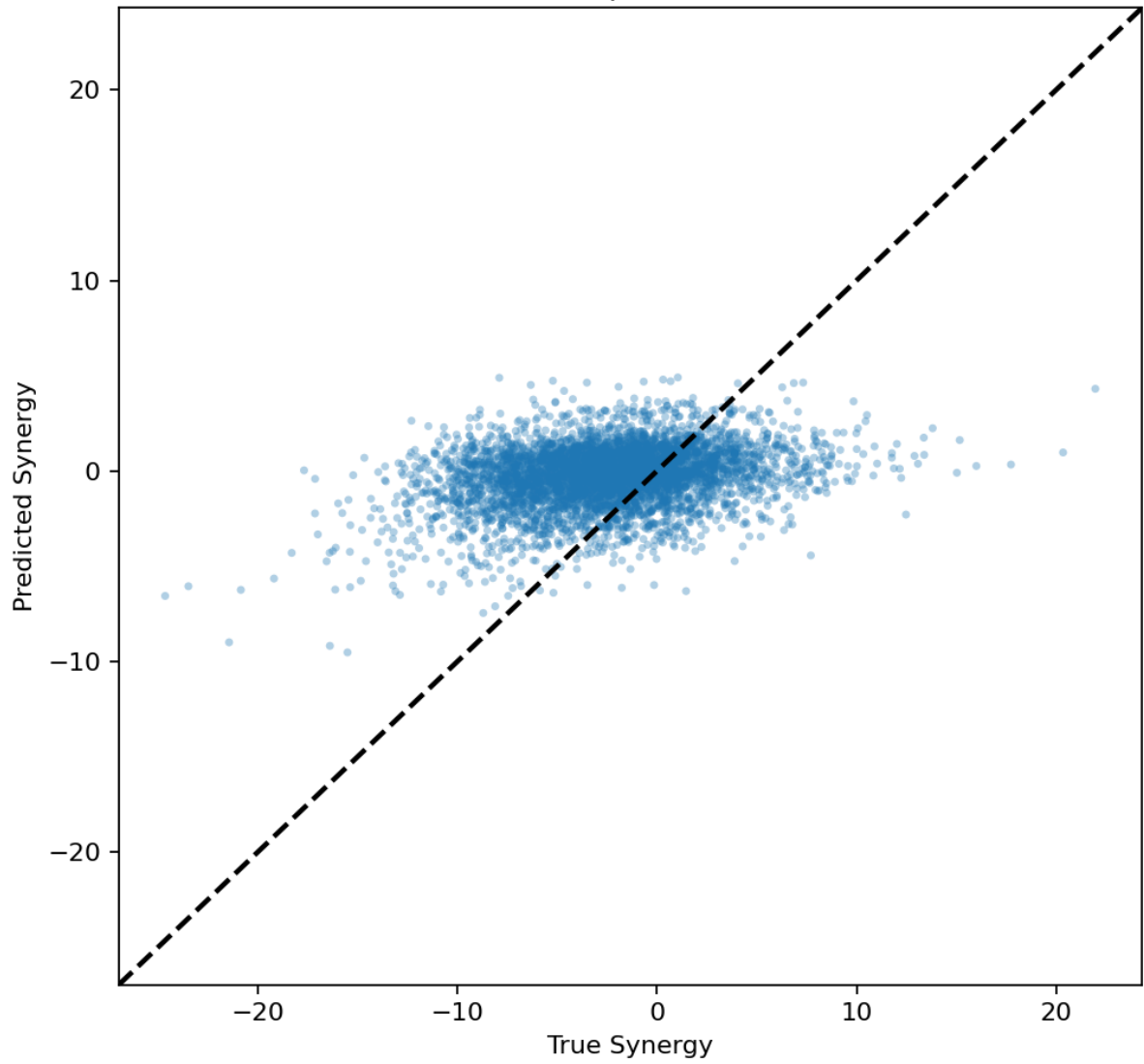
Fold 1

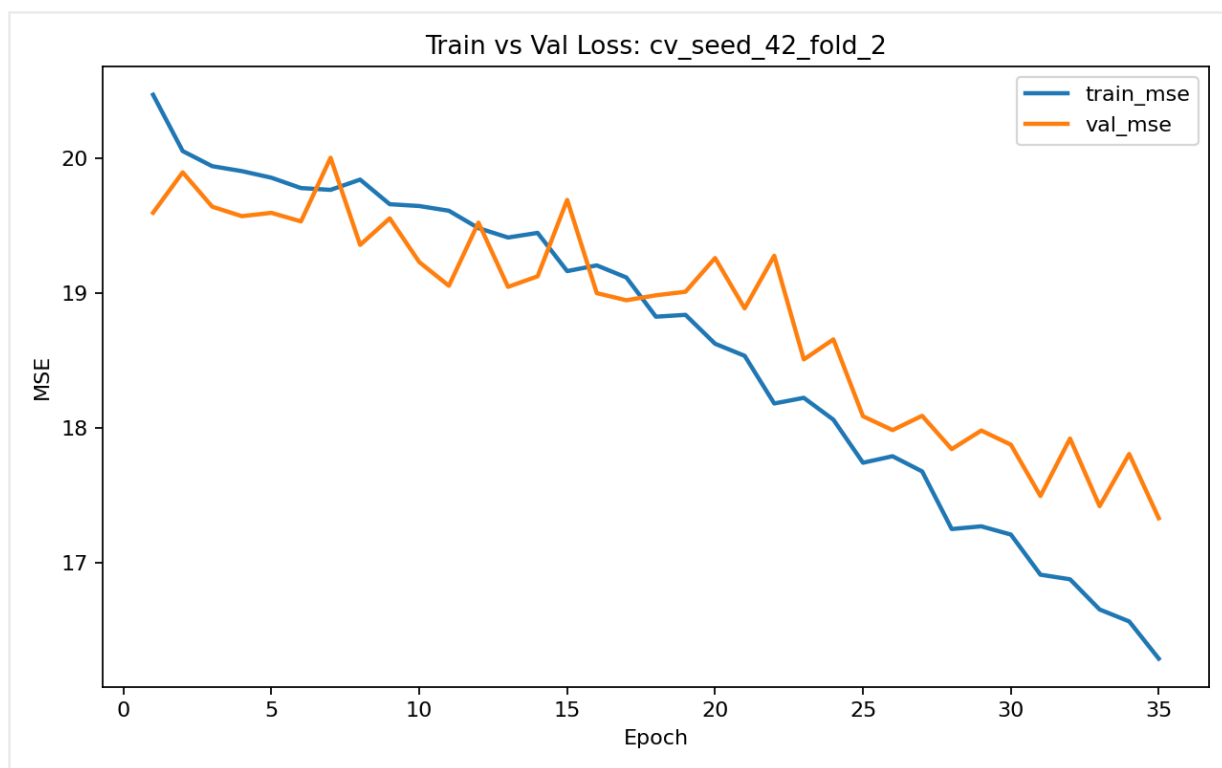




Fold 2

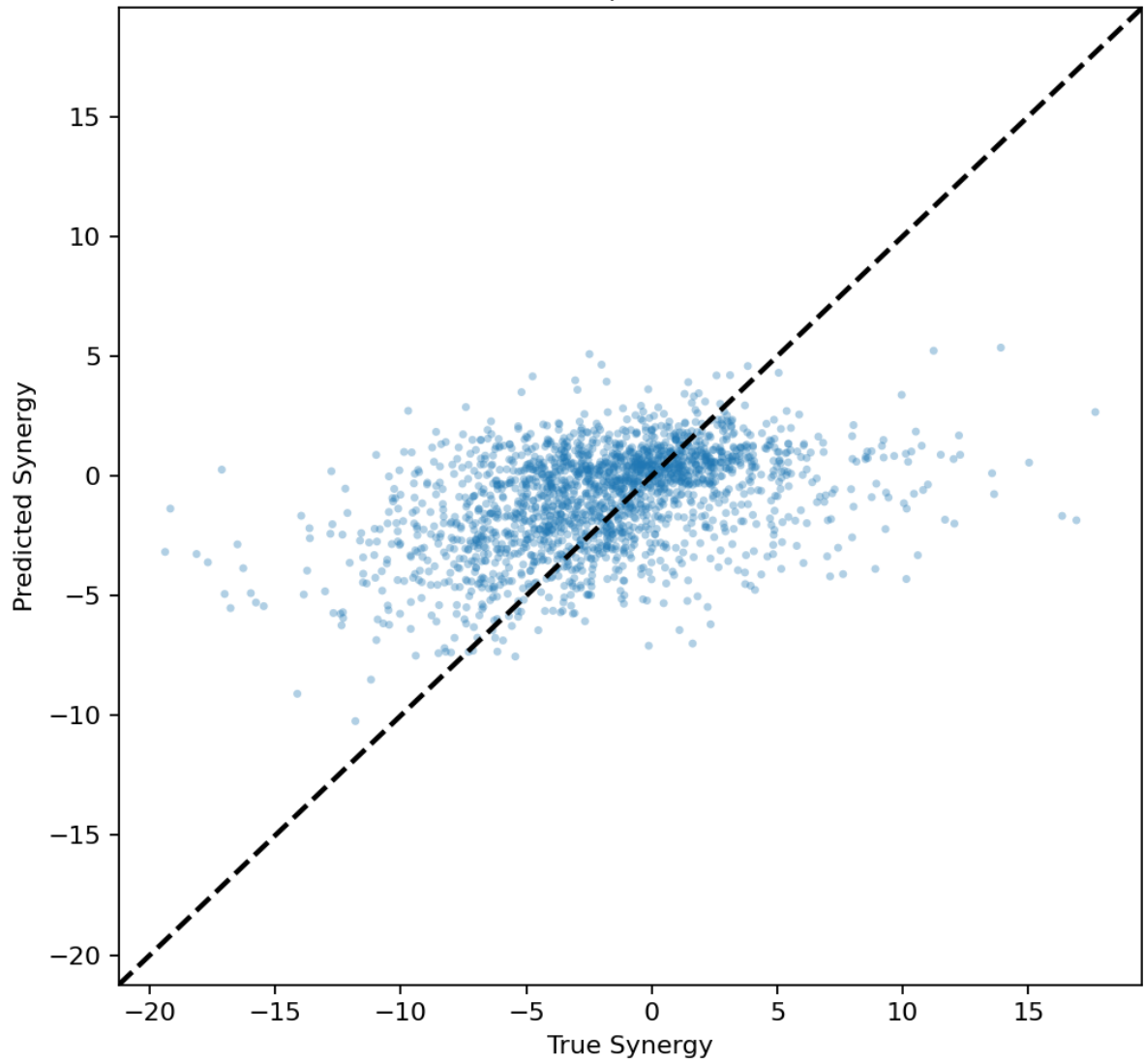
Test Regression: cv_seed_42_fold_2
MSE=23.2183 | Pearson=0.2913

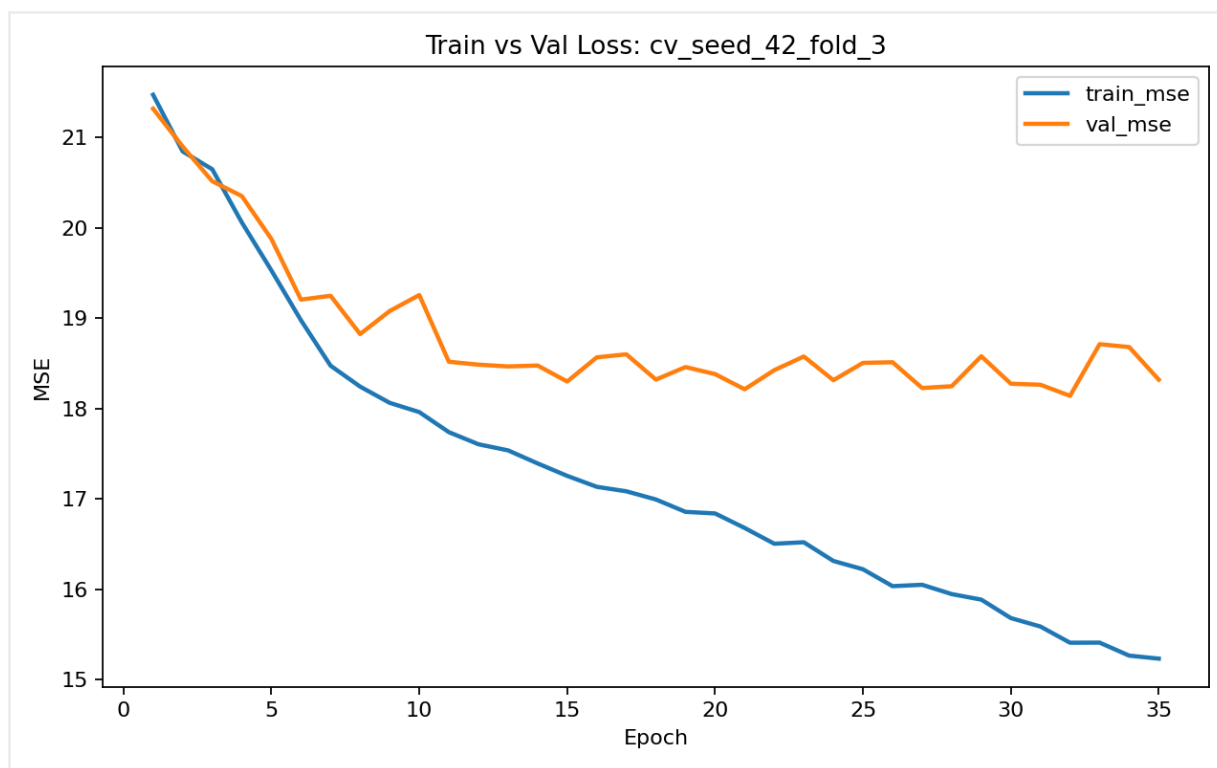




fold 3

Test Regression: cv_seed_42_fold_3
MSE=17.8099 | Pearson=0.4204

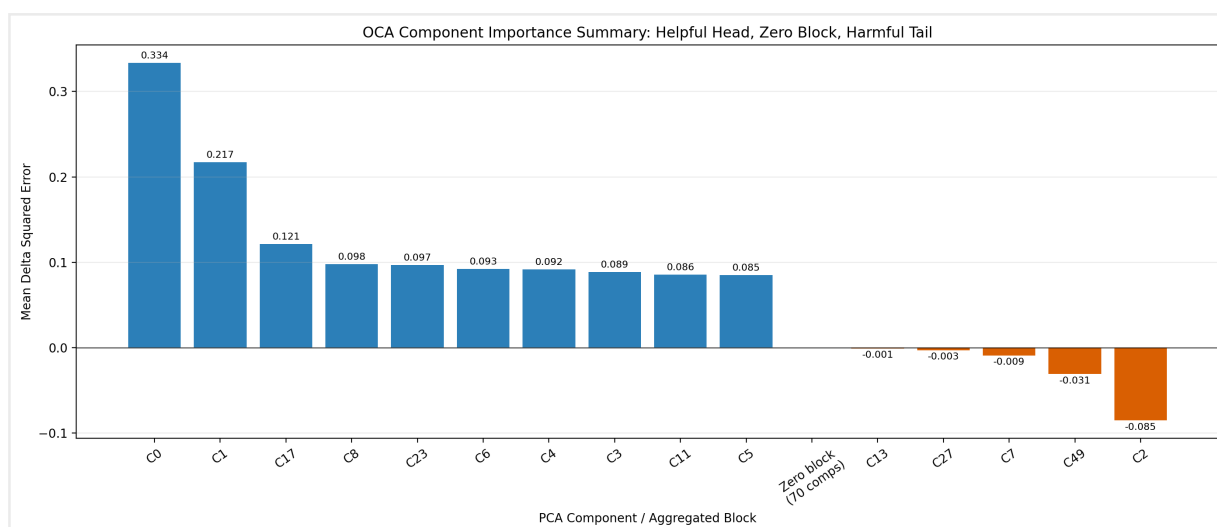




The remaining folds' plots are similar to 2 or 3, and in some cases, the train-val loss over epochs is similar to the 1st. Overall, as expected. Mean around 0. Relatively symmetrical in most cases, in some folds inclines more towards negative synergy.

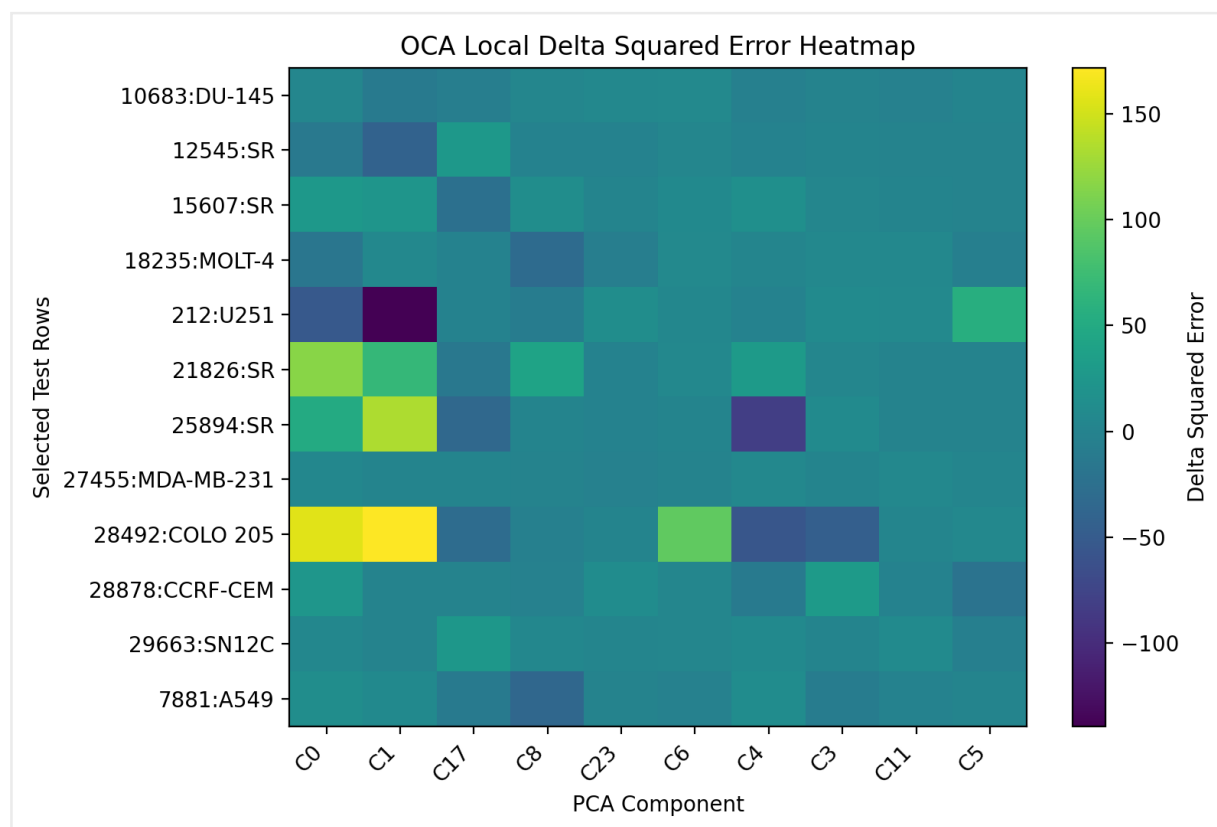
4.5 - OCA for random split run

OCA added. For baseline that uses compressed celline with random split, Global Deltas:



Seems that the third largest (or rather the genes associated with it), PC2, most negatively affects the predictions.

Interesting local cases, help us see that even though some PC AXES, like C1 do very well overall (global delta), in some specific (local) cases (samples), like 4th row from below, they perform poorly.



Highest true synergy (y_true)

- 15607:SR with y_true = 59.225925
- 21826:SR with y_true = 52.797726
- 25894:SR with y_true = 44.404881
- 7881:A549 with y_true = 43.979187
- 28492:COLO 205 with y_true = 42.779415

Lowest true synergy (y_true)

- 18235:MOLT-4 with y_true = -35.095558
- 10683:DU-145 with y_true = -29.595543
- 28878:CCRF-CEM with y_true = -27.519293
- 12545:SR with y_true = -27.480085
- 29663:SN12C with y_true = -25.055340

Largest absolute errors / residuals

- 15607:SR with |error| = 58.342276
- 7881:A549 with |error| = 44.079076

- 27455:MDA-MB-231 with |error| = 40.647340
- 212:U251 with |error| = 35.439503
- 18235:MOLT-4 with |error| = 34.899143

Some rows have more than one attribute. That's why heatmap has 12 rows that contain 15 lines.

E.g., 15607:SR is both highest y_{true} and largest error

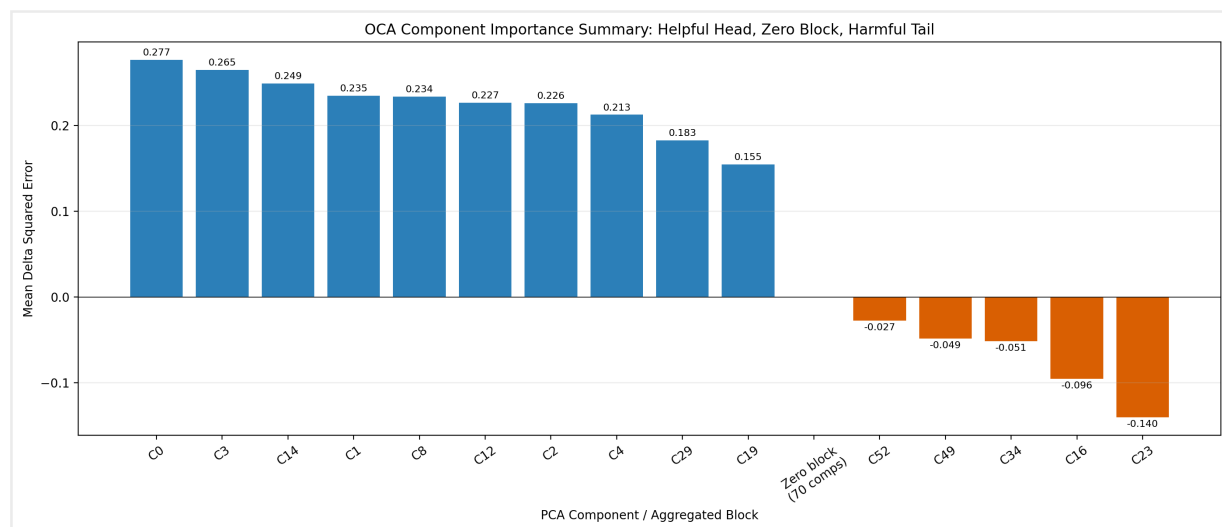
Overall conclusions:

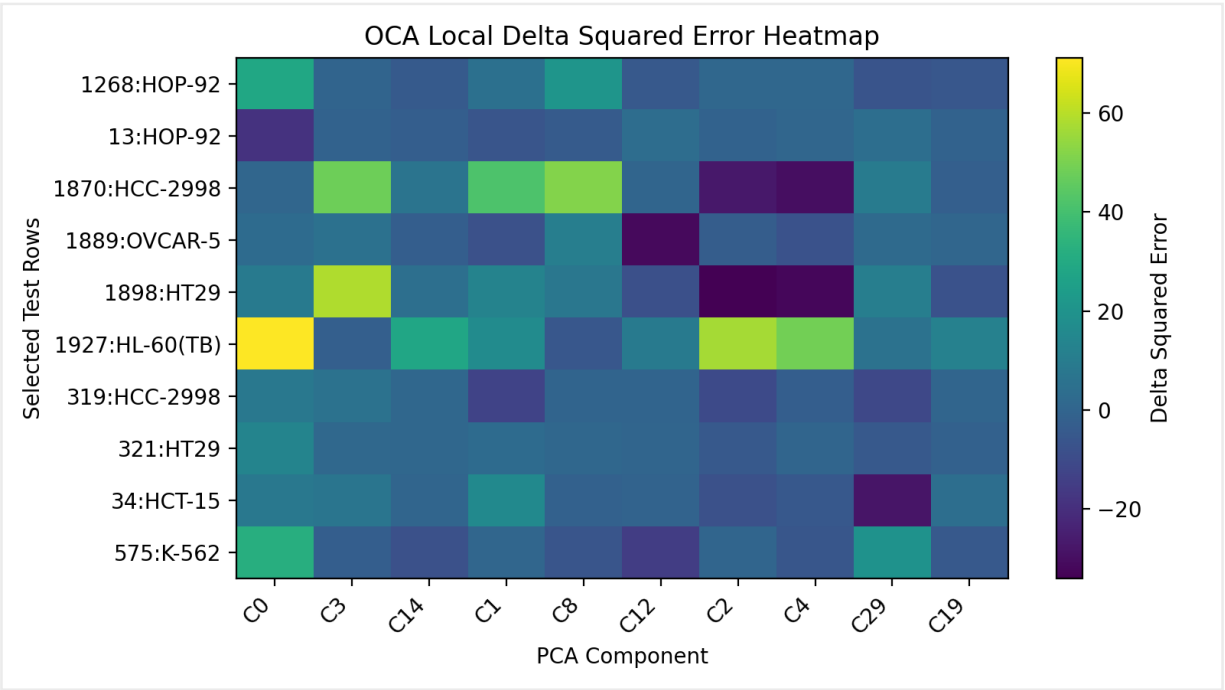
- the model relies on a small subset of PCA components
- C0 and C1 are the strongest globally helpful components
- some components are harmful despite being influential, especially C2
- many components are effectively unused / dead
- component usefulness is not uniform across samples
- therefore the cell-line signal used by the MLP is concentrated and sample-dependent

6.5 - OCA across all folds of drug and cellline selective CV run

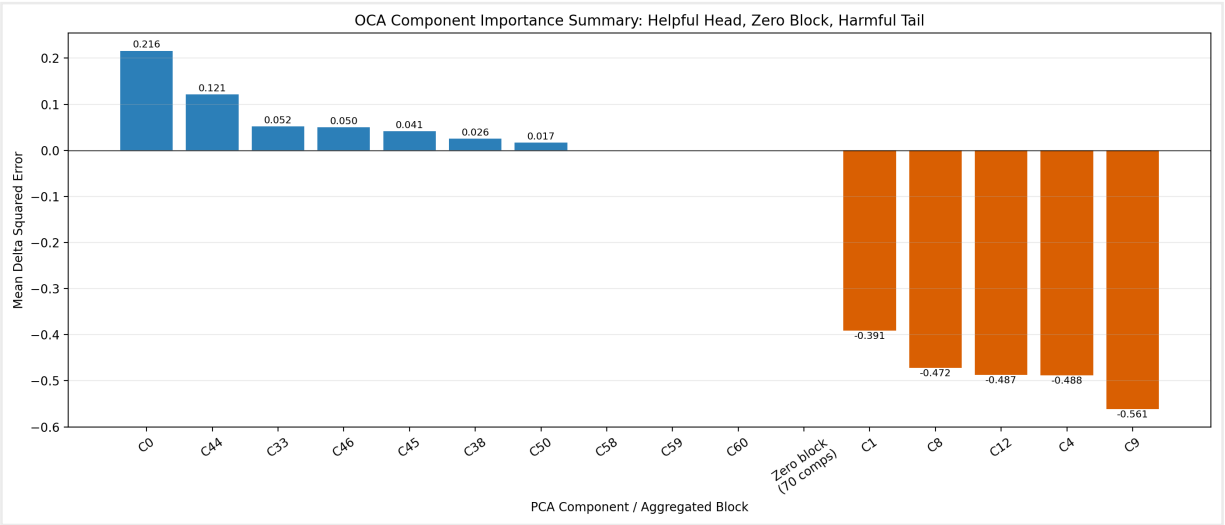
Added OCA for 10 fold CV, and summaries for all folds.

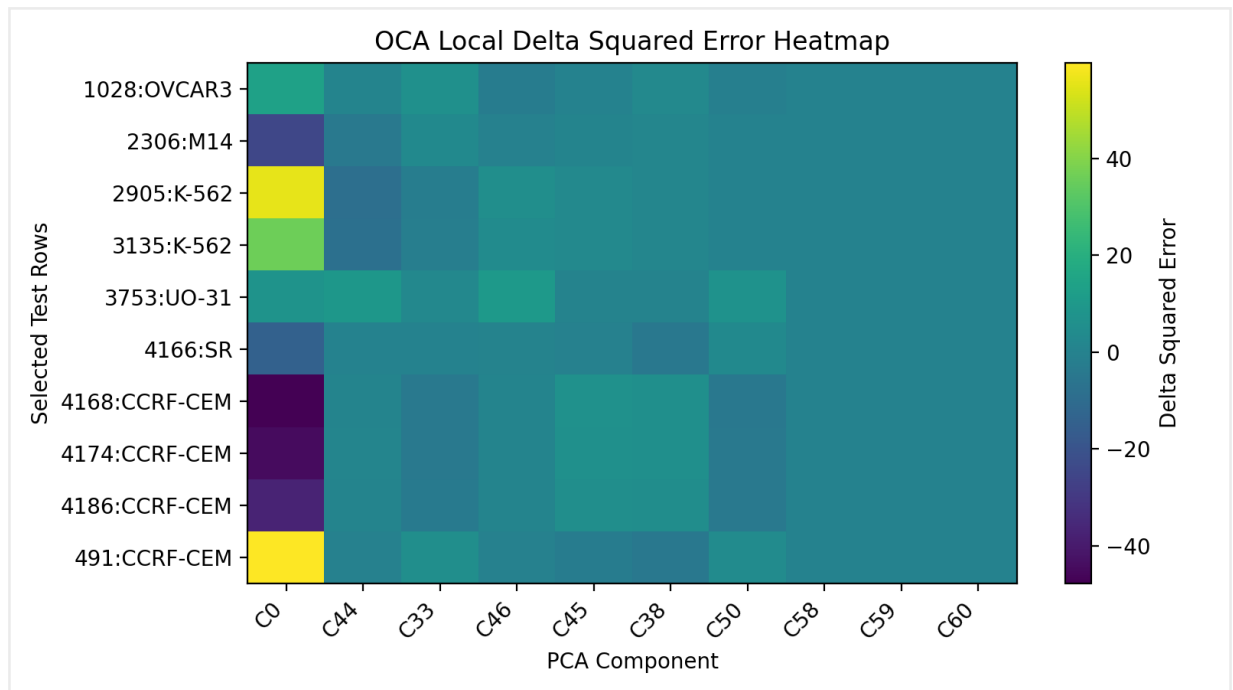
Example from fold 6 of heatmap and components OCA importance





Example from 9th fold





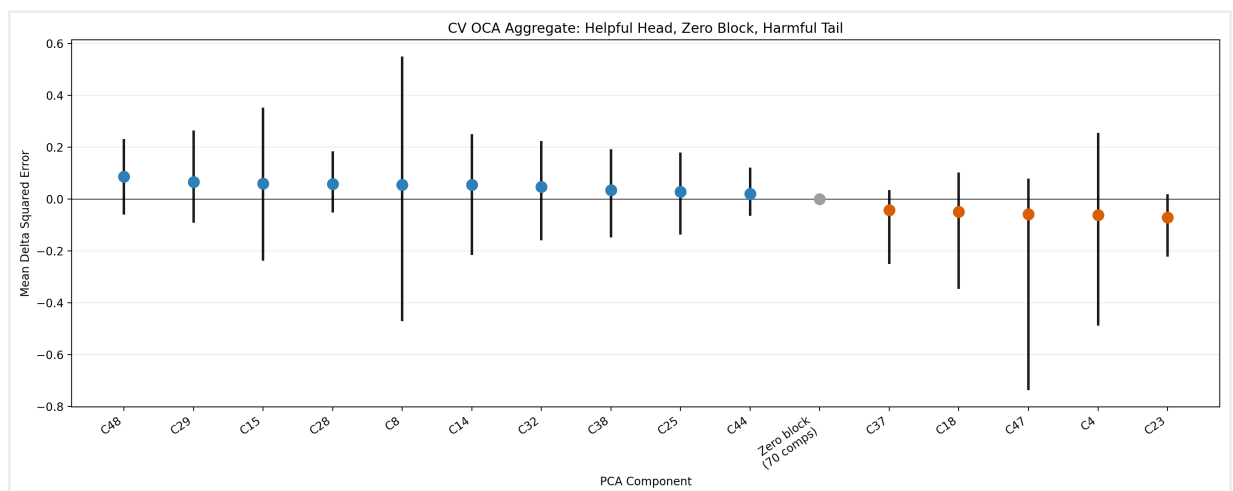
One can detect quite a difference between the folds - for 6th fold the importance of the components is more spreadout, meaning the model relies on more of them to make predictions, while for the 9th fold it mainly relies on the 0th and 5th. Also, the components for each fold are different.

For explanation regarding the heatmaps look at 4.5 notes.

I went over all of them to take the most different ones in terms of spread. There were folds with greater max, like fold 2, who's first PC passed 0.5, and the rest top 10 did far better than the other folds.

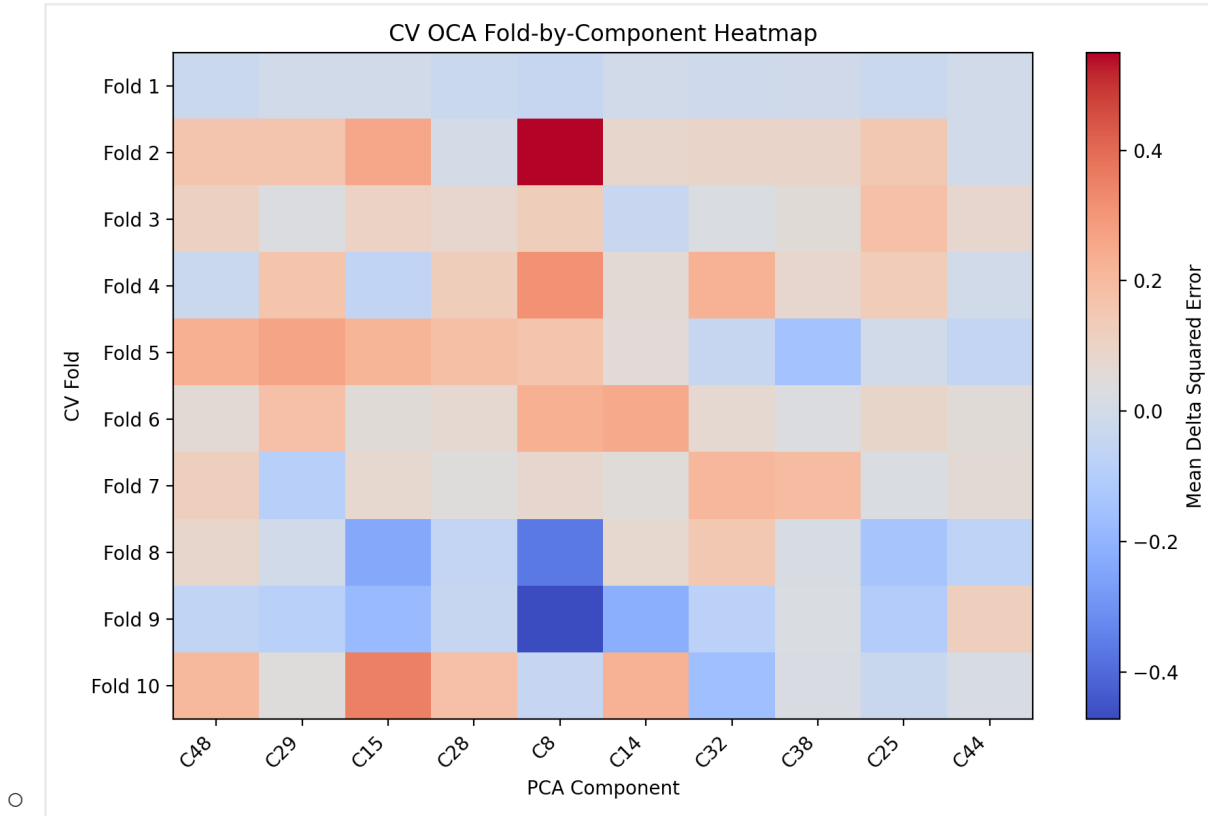
Summarizing across all folds

OCA top components

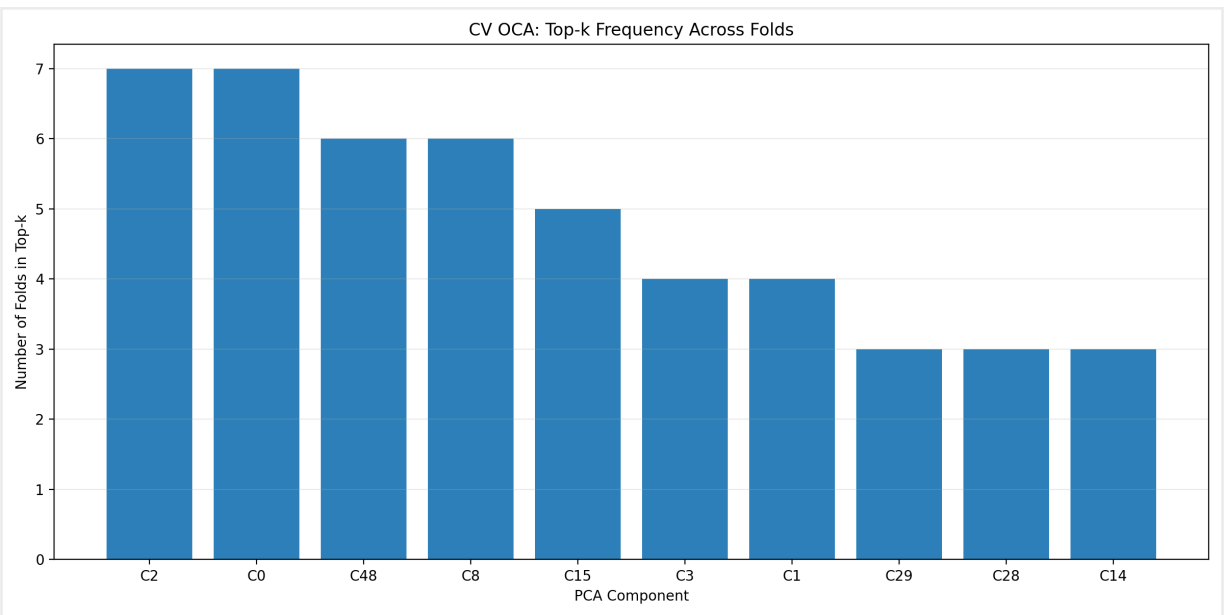


The black lines represent the maximum and minimum values the components get across all folds, the dots are the average. The components that are 0 are compressed. We can see that C8 has a fold with a maximal positive importance, ~0.6, but on average it's not the top performing component.

Another representation of the same data.



Frequencies across folds



Finally, this bar graph shows how often each component appears in the fold top-k. C2 and C0 appear 7 times at the top.

7.5 - New model with GNN/Transformer ? gene encoding instead of PCA 🌟

- ☐ Ask for context for new thread, and for GPT for the deep research.
- ☐ Message Ronen about the dataset (after I asked gpt)
- ☐ Fork chat and Let's (CODEX) plan next step architecture before implementation + (GPT) Deep research for LLM vs GNNs vs else as substitute for pca etc.
- ☐
 - How do we currently convert smiles? Do we use fingerprints?
 - Perhaps use RDKit (python package) or/and pytorch geometric.
 - Goal is to Replace existing PCA128 compression of celline with GNN, LLM or other architecture we can reverse engineer to the gene level. Use the same architecture to encode the molecules' SMILES representation. Achieve at least the same performance as the baseline model. We need this model to work best for both celline and drug smiles. List pros and cons for each.
- ☐ BEFORE ANYTHING ELSE - check if current architecture, framework, and working methods fit for later switch to 1 molecule and celline (different dataset, not drug comb) - look at Next section and make a list
- ☐ IMPLEMENTATION. Copy baseline and enhance. replace manual embeddings generation of smiles to GNN, LLM, or something (conduct deep research) that will allow reverse engineering to the gene level.

Next

Perform reverse-engineering / attribution analysis on the trained models to identify which genes or compressed components influence predictions most strongly. Repeat for molecules' encoding model.

- Could be to look at the neuron - etc., but that's problematic (for some reasons I forgot). Finally, could use attention mechanism, by lowering the

"d" factor in the Query Keys product so that the result is noiser and the sigmoid has to work harder.

Investigate whether what influences performance and how it can be improved for both a 2-drug synergy prediction and for 1 drug.

Compare improved compression to SOTA models (see papers). Potentially apply the implemented compression to their models.

If the simpler compression baselines are promising, move toward biologically informed encoders that can incorporate pathway or interaction-network structure.

Repeat the entire research for this model – test only with one drug and celline to compare predicitions.

List of things to replicate:

- Baseline model
- Train-validation-test split random + with exclusion of drugs and cellines (see 18.4.2026 notes) - with and without genes (basically recreate results/baseline_summary.csv)
- Regression plots
- OCA - highest and lowest contributing components/inputs for the folds model. Reproduce heatmaps, frequencies bar graph, and oca_cv_topk_stability.png.
- * [Move to GNNs/LLMs for embeddings encoding - next stops](#)

Summarize the findings into a clear recommendation for which cell-line representation is the best starting point for the next stage of the research/paper.